



Corrotinas

Prof. Alessandro
Brawerman

O que são Corrotinas?

- Corrotinas são uma forma leve de gerenciar concorrência em Kotlin.
- Elas permitem executar tarefas de forma assíncrona sem bloquear a execução principal do programa.
- Uma corrotina é mais eficiente em termos de recursos do que uma thread.
- Corrotinas são ideais para tarefas que podem demorar sem bloquear a interface do usuário.
 - Requests a Internet
 - Operações de I/O

Por que usar Corrotinas?

Tarefas Assíncronas

- Facilitam a execução de operações que podem ser feitas em segundo plano, como acessar uma API ou ler de um arquivo.

Eficiência

- Diferente de threads, corrotinas são muito mais leves e consomem menos recursos.

Simplificação do Código

- Evitam o uso excessivo de callbacks (funções assíncronas), resultando em código mais simples e legível.

Multiplataforma

- Funcionam em diferentes plataformas, incluindo Android, servidores e ambientes web.

Quando usar Corrotinas?

Operações de I/O

- Requests a Internet, chamadas a API, consultas de banco de dados ou leitura/escrita de arquivos.

Tarefas Longas – Computacionalmente Pesadas

- Operações que envolvem processamento intenso, como compressão de arquivos ou manipulação de imagens.

Atualização da Interface do Usuário

- Para manter a interface responsiva enquanto uma tarefa é executada em segundo plano.

Corrotinas x Threads

Threads: Pesadas, consomem mais memória e podem sobrecarregar o app se usadas em grande quantidade.

Corrotinas: São mais leves, podem ser suspensas e retomadas sem o overhead de uma thread completa.

Controle de Concorrência: Corrotinas permitem a execução concorrente de várias tarefas com menor impacto no desempenho.

runBlocking

- Função em Kotlin usada para iniciar uma corrotina de forma síncrona e bloquear a thread onde é chamada até que todas as corrotinas dentro dele sejam concluídas.
- Diferente das corrotinas regulares, que são projetadas para rodar de forma assíncrona sem bloquear a thread principal, runBlocking serve para rodar corrotinas em um estilo bloqueante.
- Útil para programas curtos ou testes onde você quer esperar explicitamente até que tudo tenha sido concluído antes de seguir em frente.

runBlocking

- Abra o site Kotlin Playground e entre com o programa abaixo
 - <https://play.kotlinlang.org/>

```
import kotlinx.coroutines.*
```

```
fun main() {  
    println("Iniciando programa...")  
    runBlocking {  
        println("Iniciando tarefa pesada ...")  
        delay(1000) // Simula uma tarefa de 1 segundo  
        println("Fim da tarefa pesada")  
    }  
    println("Fim do programa")  
}
```

runBlocking

- A chamada `runBlocking` é síncrona, o bloco bloqueia a thread e não vai retornar até que todo o trabalho no bloco esteja concluído.
- `delay(1000)`
 - Suspende a thread por 1 segundo.

Quando evitar runBlocking?

- Em apps Android
 - O runBlocking geralmente não deve ser usado em código Android em produção, especialmente na thread principal, pois bloquearia a interface do usuário.
 - Em vez disso, em Android, corrotinas devem ser executadas de forma assíncrona usando launch e async.
- Em qualquer código que precise ser altamente responsivo
 - Se o código precisa ser executado sem bloquear a interface ou a execução de outros processos, evite runBlocking e use soluções mais adequadas para a execução assíncrona.

suspend

```
import kotlinx.coroutines.*

fun main() {
    println("Iniciando programa ...")
    runBlocking {
        println("Iniciando tarefa pesada ...")
        heavyTask()
    }
    println("Fim do programa")
}

fun heavyTask() {
    delay(1000)
    println("Fim da tarefa pesada")
}
```

suspend

- Note o erro no código anterior informando que `delay` é uma função de suspensão e deveria ser chamada apenas dentro de uma outra função de suspensão ou corrotina.
- Funções suspensas são aquelas que podem ser pausadas e retomadas sem bloquear a thread.
- Uma função de suspensão é como uma função normal, mas pode ser suspensa e retomada novamente mais tarde.
- Para fazer isso, as funções de suspensão só podem ser chamadas usando outras funções desse tipo que disponibilizem esse recurso.
 - Precisa estar dentro de um bloco de corrotina (`launch`, `async`, ou `runBlocking`).
- Ela não cria uma nova thread.

suspend

```
import kotlinx.coroutines.*

fun main() {
    println("Iniciando programa ...")
    runBlocking {
        println("Iniciando tarefa pesada ...")
        heavyTask()
    }
    println("Fim do programa")
}

suspend fun heavyTask() {
    delay(1000)
    println("Fim da tarefa pesada")
}
```

suspend

```
import kotlinx.coroutines.*
```

```
suspend fun carregarDados(): String {  
    delay(2000) // Simula o tempo de uma requisição de rede  
    return "Dados carregados com sucesso!"  
}
```

```
fun main(){  
    runBlocking {  
        println("Carregando dados...")  
        val resultado = carregarDados()  
        println(resultado)  
    }  
}
```

suspend

```
import kotlin.system.*
import kotlinx.coroutines.*

fun main() {
    println("Iniciando programa ...")
    val time = measureTimeMillis {
        runBlocking {
            println("Iniciando tarefas pesadas ...")
            heavyTask1()
            heavyTask2()
        }
    }
    println("Execution time: ${time / 1000.0} seconds")
    println("Fim do programa")
}
```

suspend

```
suspend fun heavyTask1() {  
    delay(1000)  
    println("Fim da tarefa pesada 1")  
}
```

```
suspend fun heavyTask2() {  
    delay(1000)  
    println("Fim da tarefa pesada 2")  
}
```

- Veja que o tempo de execução é um pouco mais que 2 segundos.
- Guarde este número.